

De un modelo de IA a una API en producción con Cloud Run





Diana Sánchez Ordóñez
Software Developer

DESARROLLO

Desarrollo de Software

Más de 4 años construyendo aplicaciones web y APIs end to end

Nodejs

Shopify

Go

Python

React

Typescript

CLOUD & IA

Desarrollo APIs con FastAPI y trabajo en AWS y GCP, con una Maestría en Ciencia de Datos e IA.

AWS

GCP



Qué vamos a construir hoy ?



1. Modelo

Elegimos un modelo pre-entrenado de análisis de sentimientos, sin entrenar nada nosotros.



2. API

Lo envolvemos en un endpoint FastAPI: POST /predict recibe texto y devuelve el resultado.



3. Contenedor

Empaquetamos la API y sus dependencias en una imagen Docker reproducible.



4. Despliegue

Subimos la imagen a Artifact Registry y la desplegamos en Cloud Run.

Caso de Uso: Clasificación de sentimientos

Dado un texto en lenguaje natural, el modelo determina si expresa una opinión positiva o negativa, junto con un nivel de confianza. Es uno de los casos de uso de NLP más comunes en la industria.

- Monitoreo de redes sociales y marca
- Análisis de reseñas de productos
- Priorización de tickets de soporte
- Encuestas de satisfacción a gran escala

"¡Me encantó el servicio, todo excelente!"

 **POSITIVO · 98%**

"El producto llegó dañado y con retraso."

 **NEGATIVO · 95%**

El modelo



```
distilbert-base-uncased-  
finetuned-sst-2-english
```

67M

parámetros

~260 MB

tamaño en disco

2 clases

POSITIVE / NEGATIVE

<https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english>

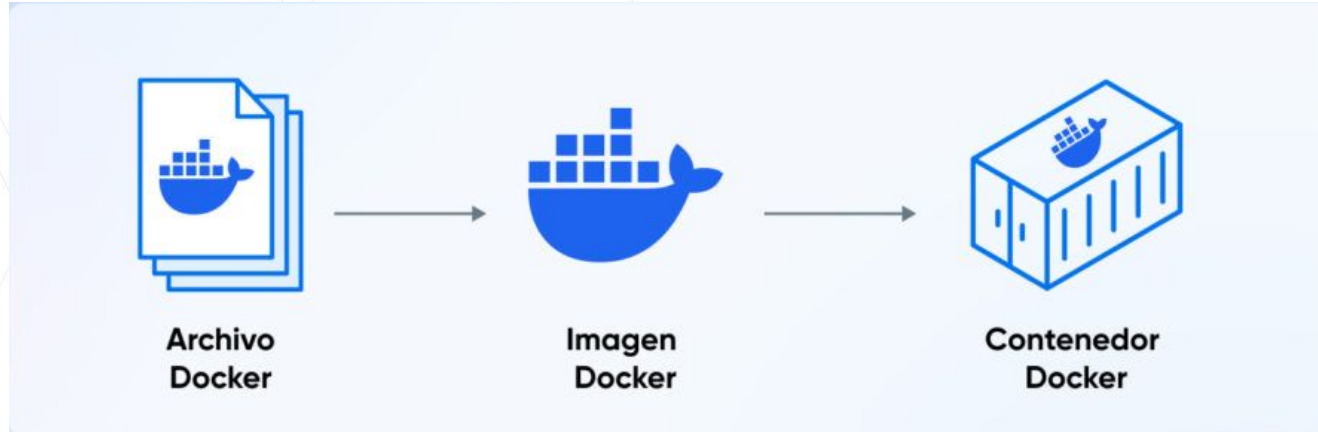


Máxima potencia

Fase 1



Docker



Docker permite empaquetar una aplicación con todo lo que necesita para funcionar y ejecutarla de forma consistente en diferentes entornos.

Código



Docker

- Construir la imagen

```
docker build -t taller-sentimientos .
```

- Verificar la imagen

```
docker images
```

- Ejecutar el contenedor

```
docker run -p 8080:8080 taller-sentimientos
```

- Descargar la imagen

```
docker save -o taller-sentimientos.tar  
taller-sentimientos:latest
```

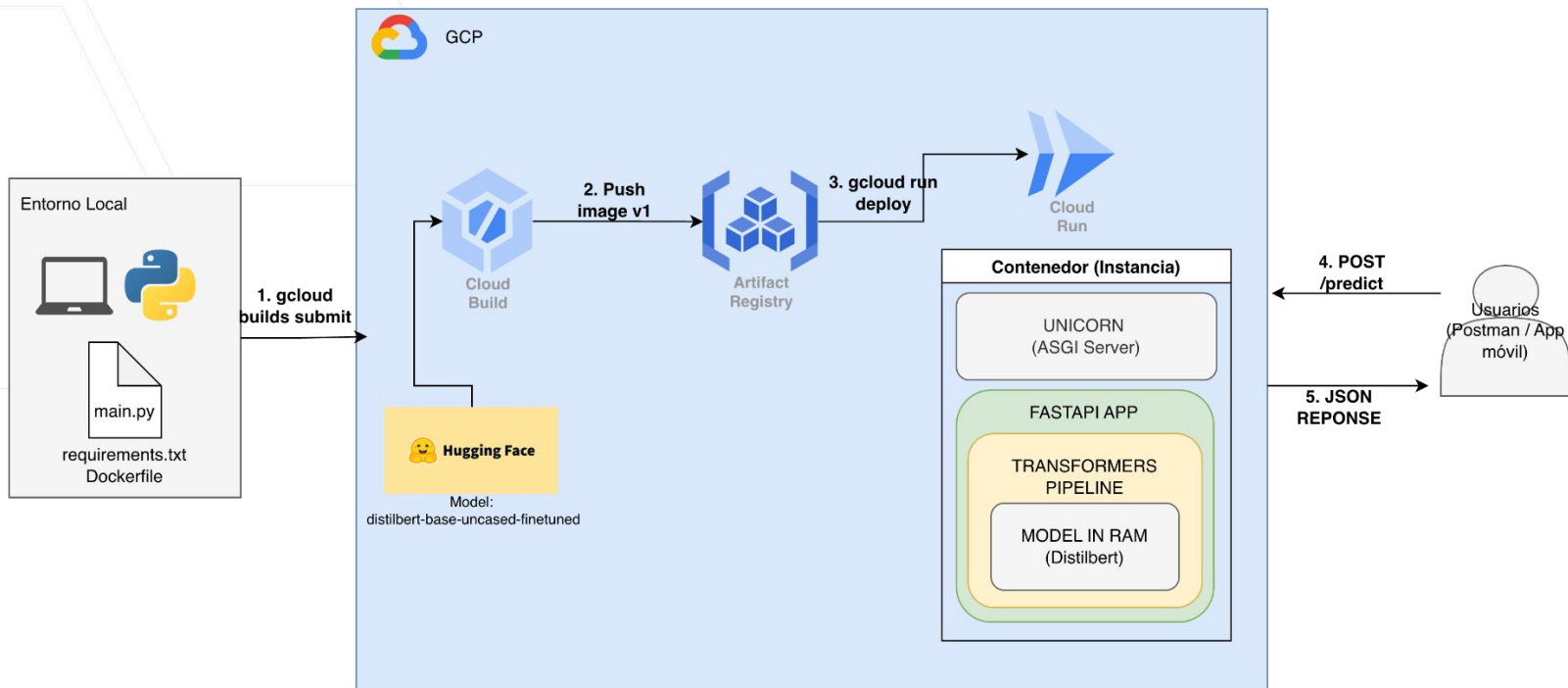
- Importar la image

```
docker load -i taller-sentimientos.tar
```

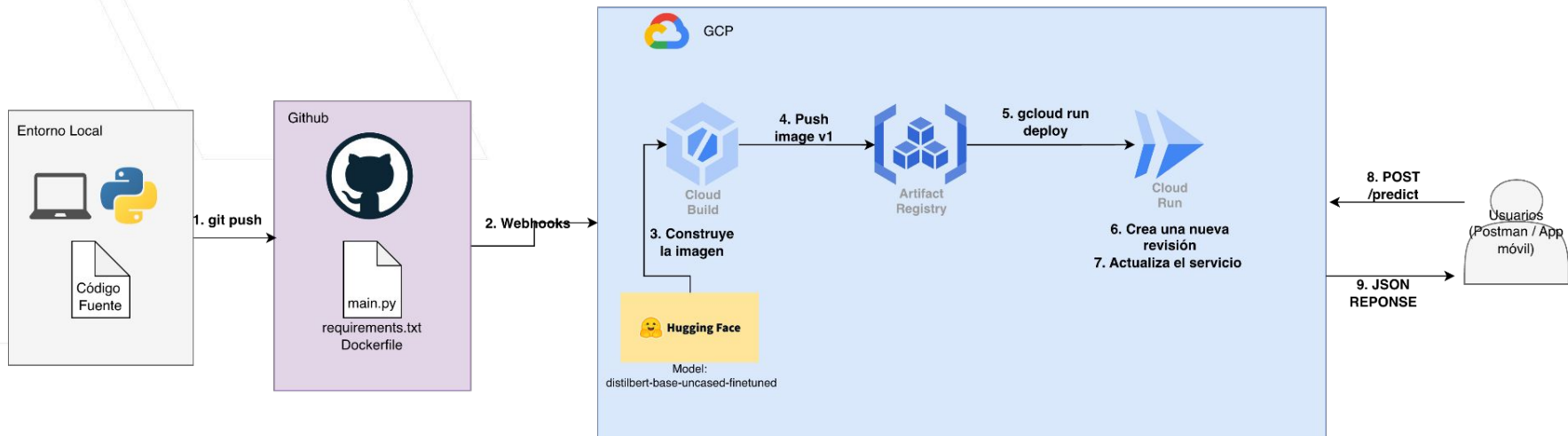
Arquitectura

FLUJO DESPLIEGUE (CI/CD) us-central1

ARQUITECTURA DE EJECUCIÓN (RUNTIME)



Despliegue automático desde GitHub



Cloud Run no elimina los contenedores cuando desplegamos desde GitHub; simplemente abstrae el proceso de construir, almacenar y desplegar la imagen.

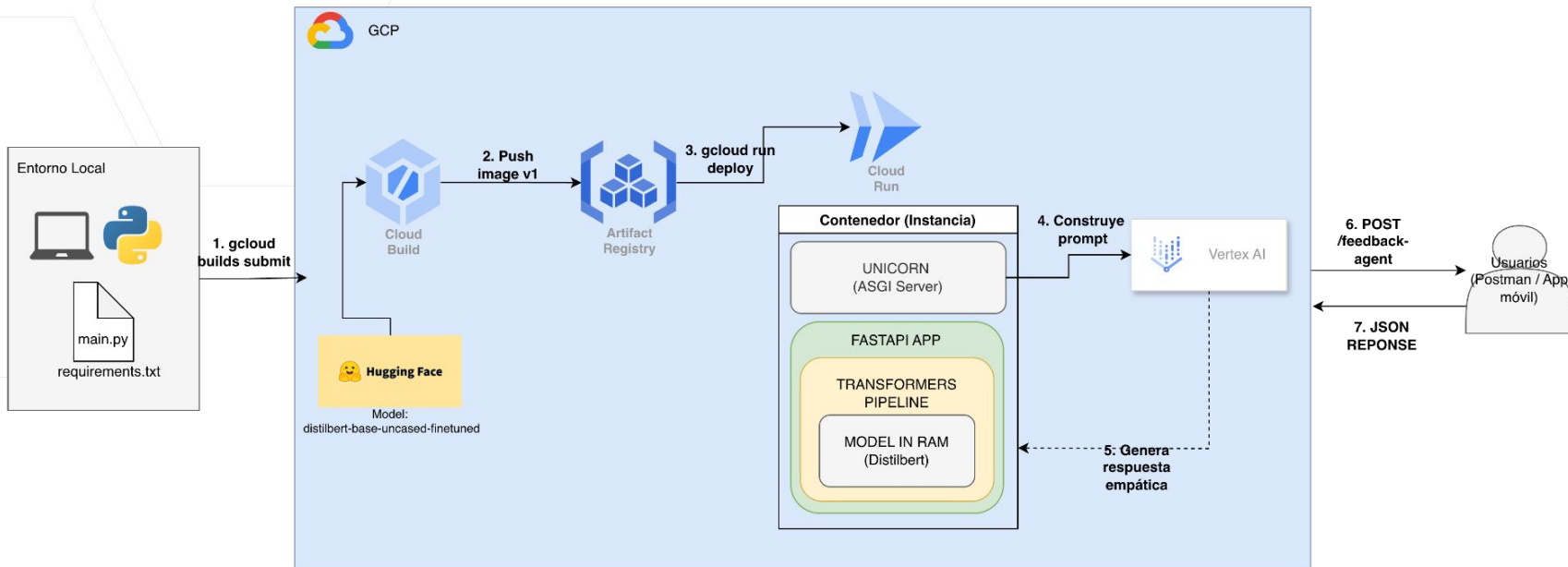
Fase 2: Vertex AI



Vertex AI

FLUJO DESPLIEGUE (CI/CD) us-central1

ARQUITECTURA DE EJECUCIÓN (RUNTIME)



GCP	AWS	Notas
Cloud Run	AWS App Runner	Ambos son serverless de contenedores con escalado a cero. Fargate (ECS/EKS) es la alternativa más "manual" — App Runner es la comparación más directa.
Artifact Registry	Amazon ECR (Elastic Container Registry)	Casi 1:1 — registro de imágenes Docker privado.
Cloud Build	AWS CodeBuild	Ambos compilan/empaquetan en la nube sin infraestructura propia.
Cloud Logging	Amazon CloudWatch Logs	Equivalente directo.
Cloud Monitoring	Amazon CloudWatch (Metrics)	Mismo concepto — dashboards y métricas de recursos.
Vertex AI (Endpoints / Model Registry)	Amazon SageMaker	SageMaker cubre entrenar, registrar y desplegar modelos propios — el equivalente más completo.
Vertex AI + Gemini	Amazon Bedrock	Bedrock te da acceso a modelos fundacionales (Claude, Llama, Titan) sin desplegar nada, igual que Gemini vía Vertex AI.
IAM	AWS IAM	Mismo concepto de roles y políticas, sintaxis distinta.



Cuánto cuesta Cloud Run en producción

Servicio	Free tier mensual	Precio después
Cloud Run (CPU)	180,000 vCPU-segundo	\$0.000024 / vCPU-s
Cloud Run (memoria)	360,000 GiB-segundo	\$0.0000025 / GiB-s
Cloud Run (requests)	2,000,000 requests	\$0.40 / millón
Artifact Registry	0.5 GB de almacenamiento	\$0.10 / GB / mes
Cloud Build	2,500 minutos de build	\$0.006 / minuto

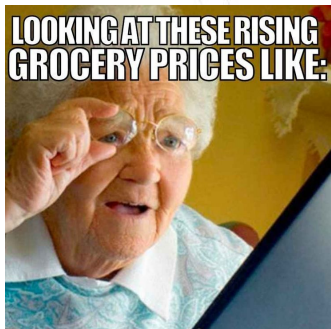
Escenario A: cada request tarda 100ms en responder $\rightarrow 1000 \times 0.1s = 100$ vCPU-segundos consumidos

Escenario B: cada request tarda 2 segundos (por ejemplo, porque llama al modelo de IA) $\rightarrow 1000 \times 2s = 2000$ vCPU-segundos consumidos

<https://cloud.google.com/run/pricing>



Gemini se paga por tokens, no por servidor



Modelo	Input / 1M tokens	Output / 1M tokens
Gemini 2.5 Flash-Lite	\$0.10	\$0.40
Gemini 2.5 Flash (el nuestro)	\$0.30	\$2.50
Gemini 2.5 Pro	\$1.25	\$10.00

Ejemplo real: un comentario de ~30 palabras (~40 tokens de entrada) + una respuesta de 3 líneas (~60 tokens de salida) con Gemini 2.5 Flash cuesta aproximadamente **\$0.00016 USD** por interacción.

<https://docs.cloud.google.com/gemini-enterprise-agent-platform/resources/locations?hl=es-419#global>

Preguntas



